SaúdeCerta — Sistema de Gestão de Clínica

Entrega 2: Projeto Arquitetural

IFPA · Tecnologia em Análise e Desenvolvimento de Sistemas · TEADS0347 – Desenvolvimento de Aplicações Multicamadas · Turma NOITE C795NL · 2026.2 · Docente: Márcio Goes do Nascimento

Equipe: [nomes dos integrantes] · Repositório Git: [link do repositório da equipe]

1. Drivers e cenários de qualidade

Ordem de prioridade dos drivers (E1): 1) evolução e modificabilidade, 2) simplicidade e manutenção, 3) desempenho e disponibilidade; segurança e LGPD entram como restrição obrigatória.

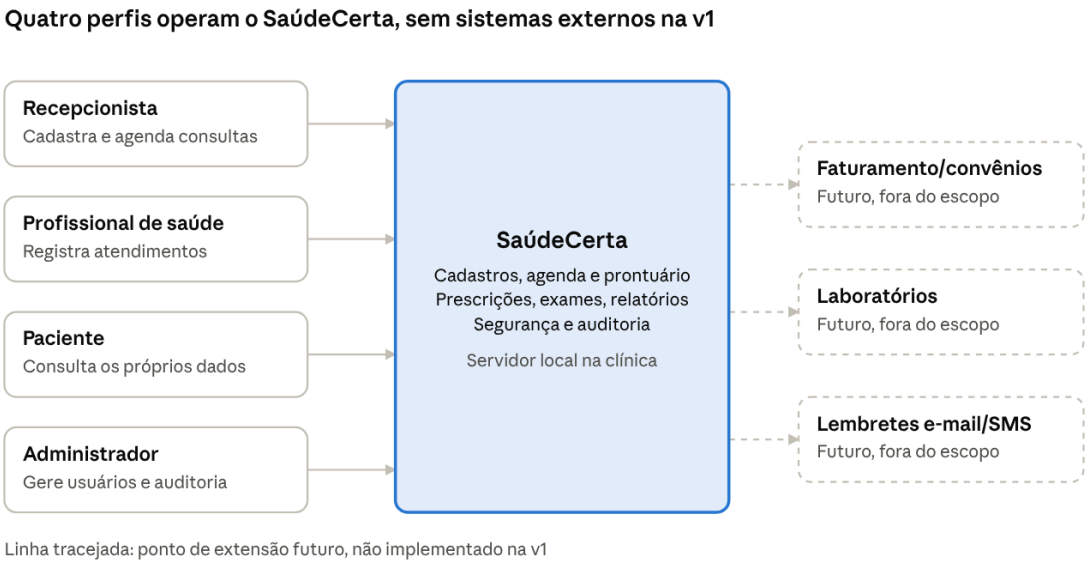
ID	Atributo	Situação	O sistema deve	Meta
RNF01	Modificabilidade	A equipe inclui o módulo de faturamento (futuro)	Aceitar o módulo novo sem mudar agenda e prontuário	Até 5 dias; nenhuma alteração no domínio de agenda e prontuário
RNF02	Manutenibilidade	Um novo integrante clona o repositório	Subir localmente com H2 e dados de exemplo	Até 30 min seguindo o README
RNF03	Segurança	Recepcionista, profissional sem vínculo ou paciente tenta abrir um prontuário que não pode	Responder 403 sem conteúdo clínico e auditar como NEGADO	100% das tentativas indevidas bloqueadas
RNF04	Auditabilidade	Alguém abre um prontuário	Registrar usuário, paciente, data/hora, ação e resultado	Visível ao ADMIN em até 1 min
RNF05	Desempenho	Recepcionista busca paciente por nome; profissional abre prontuário	Responder paginado e com índices, com 20 profissionais, 10 mil pacientes e 50 mil consultas	p95 abaixo de 2 s
RNF06	Disponibilidade	Falha do processo no horário de	Reiniciar sozinho, expor healthcheck (Spring	99% ou mais por mês no horário. Falha de

ID	Atributo	Situação	O sistema deve	Meta
		funcionamento (12 h/dia)	Actuator), usar transações curtas e timeouts	hardware: recuperação manual
RNF07	Usabilidade	Recepcionista marca uma consulta	Oferecer fluxo curto, com aviso imediato de horário ocupado	Até 1 min e 5 cliques a partir da tela inicial
RNF08	Credenciais	Revisor inspeciona código e banco	Guardar senhas só com BCrypt, nunca em log	Zero senhas em texto
RNF09	Recuperabilidade	Perda do disco do servidor	Restaurar o banco a partir do backup diário em outro dispositivo	Perda máxima de 24 h; restauração testada todo mês

2. Diagramas C4

2.1 Nível 1: Contexto

Elemento	Tipo	Descrição
Recepcionista (RECEPCAO)	Pessoa	Cadastra pacientes, marca, remarca e cancela consultas. Sem acesso clínico.
Profissional de saúde (PROFISSIONAL)	Pessoa	Registra atendimentos, adendos, prescrições e exames dos pacientes com os quais tem vínculo.
Paciente (PACIENTE)	Pessoa	Consulta os próprios dados, agendamentos e prontuário, somente leitura.
Administrador (ADMIN)	Pessoa	Gerencia usuários, disponibilidade e especialidades; consulta auditoria e relatórios; lê prontuários.
SaúdeCerta	Sistema	Gestão de cadastros, agenda, prontuário, prescrições e exames, relatórios, segurança e auditoria.
Faturamento e convênios, laboratórios, lembretes	Sistemas externos (futuros, fora do escopo)	Pontos de extensão; não existem na v1.



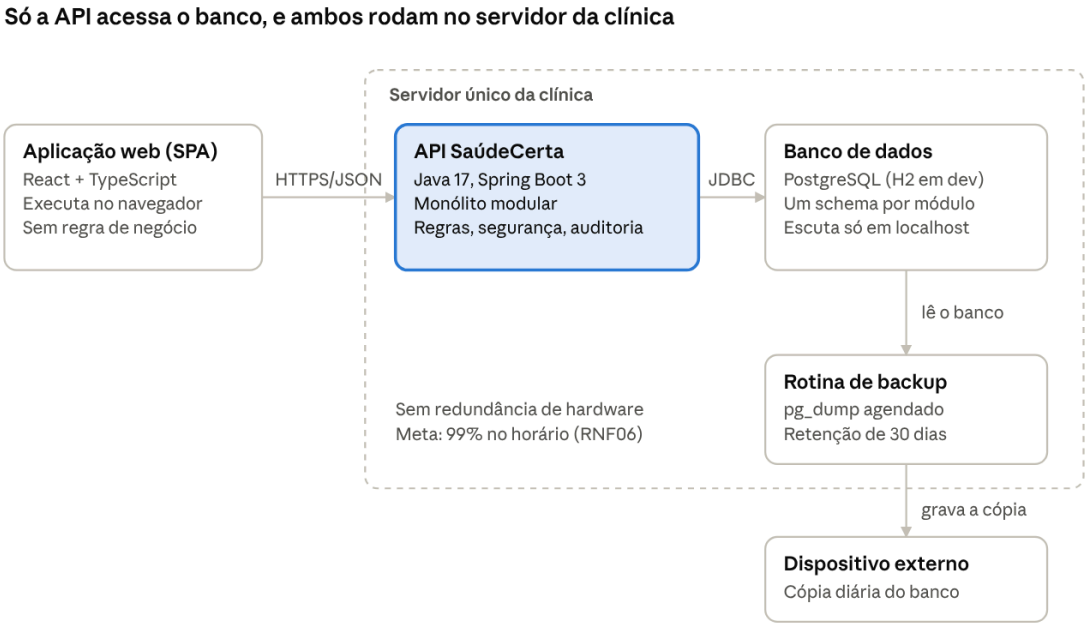
C4 nível 1, contexto · 4 pessoas, 1 sistema, 3 externos futuros

O diagrama resume a tabela: os quatro perfis usam o sistema dentro da clínica, e os sistemas externos aparecem tracejados por serem extensões futuras.

2.2 Nível 2: Contêineres

Contêiner	Tecnologia	Responsabilidade	Comunicação
Aplicação web (SPA)	React + TypeScript (Vite), no navegador	Telas e experiência de uso; sem regra de negócio	HTTPS + JSON (REST) com a API
API SaúdeCerta	Java 17, Spring Boot 3 (monólito modular)	Casos de uso, regras, segurança, auditoria e relatórios	JDBC com o banco
Banco de dados	PostgreSQL (H2 em desenvolvimento), um schema por módulo	Cadastros, agenda, prontuário, prescrições, exames e auditoria	Acessado só pela API
Rotina de backup	Script agendado (pg_dump)	Backup diário copiado para outro dispositivo, com retenção de 30 dias	Lê o banco; grava no dispositivo externo

Implantação: servidor único na clínica, com API e banco em processos separados. O banco escuta só em localhost e o cliente nunca o acessa direto. O tráfego usa HTTPS.



C4 nível 2, contêineres · aplicação web, API, banco PostgreSQL e rotina de backup

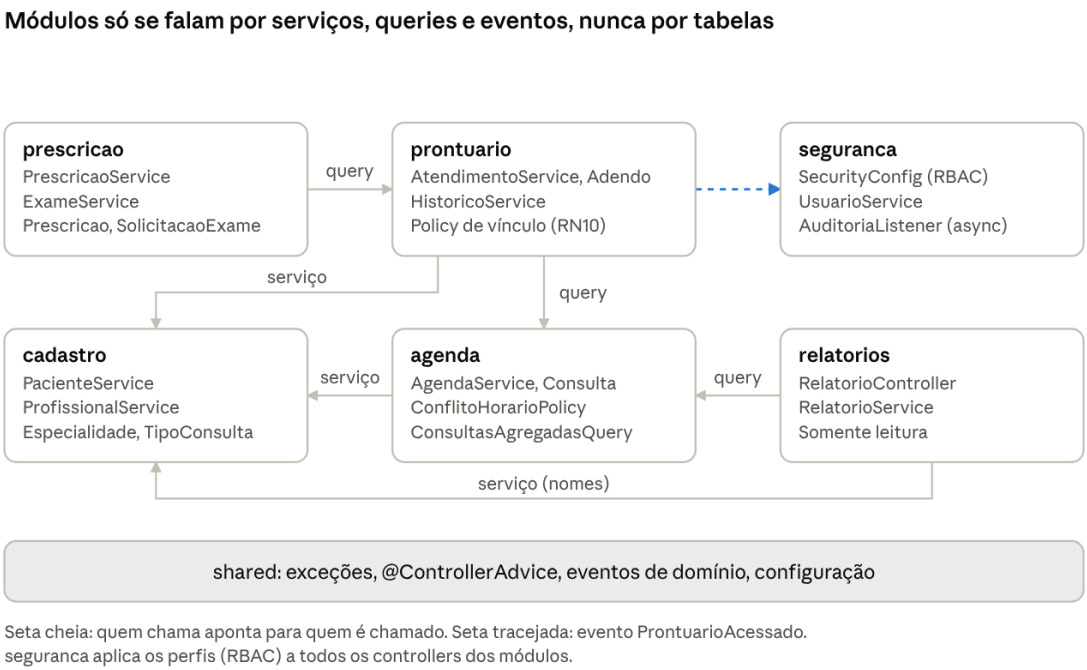
O navegador só conversa com a API; o banco não é acessível por fora do servidor, e o backup é a única saída de dados para outro dispositivo.

2.3 Nível 3: Componentes da API

Módulo	Componentes principais	Depende de
cadastro	PacienteController e PacienteService; ProfissionalService, EspecialidadeService, TipoConsultaService; Paciente, Profissional e portas de repositório	shared
agenda	ConsultaController, DisponibilidadeController, AgendaService, Consulta, ConflitoHorarioPolicy, CalculadoraHorariosLivres, ConsultasDoProfissionalQuery, ConsultasAgregadasQuery	cadastro (via serviço), shared
prontuario	ProntuarioController, AtendimentoService, HistoricoService, Atendimento, Adendo, VinculoProfissionalPacientePolicy, publicador do evento ProntuarioAcessado, AtendimentoQuery	agenda (via ConsultasDoProfissionalQuery), cadastro, shared

Módulo	Componentes principais	Depende de
prescricao	PrescricaoController, ExameController, PrescricaoService, ExameService, Prescricao, ItemPrescricao, SolicitacaoExame	prontuario (via AtendimentoQuery), shared
relatorios	RelatorioController, RelatorioService	agenda (via ConsultasAgregadasQuery), cadastro (nomes), shared
seguranca	SecurityConfig (sessão, BCrypt, CSRF), UsuarioService, regras RBAC, AuditoriaListener (assíncrono), AuditoriaService, AuditoriaController (só ADMIN)	shared (eventos)
shared	Exceções, @ControllerAdvice, eventos de domínio, configuração, utilitários	nenhum

Regras de dependência: (1) módulos só se falam por serviços de aplicação, interfaces de consulta (Query) ou eventos, nunca por tabelas ou repositórios do outro; (2) referências entre módulos são por ID; (3) não há dependência cíclica. A verificação é feita em revisão de código, como previsto na E1.



C4 nível 3, componentes da API · 6 módulos, shared e 1 evento

O único acoplamento entre prontuário e segurança é o evento; as demais dependências seguem uma única direção, sem ciclos.

Fluxo 'abrir prontuário': o controller recebe a requisição → o perfil é checado → o HistoricoService verifica o vínculo (RN10) ou a propriedade (RN12) → carrega atendimentos, adendos, prescrições e exames → publica ProntuarioAcessado → o AuditoriaListener grava o registro de forma assíncrona → resposta. Se o acesso for negado, o mesmo evento é publicado com resultado NEGADO e a resposta é 403.

3. Camadas e componentes

Camada	Responsabilidade	Exemplo (módulo agenda)	O que não pode fazer
Apresentação (api)	Receber HTTP, validar entrada, converter para DTO	ConsultaController, MarcarConsultaRequest	Conter regra de negócio
Aplicação (application)	Orquestrar caso de uso, transação e chamadas a outros módulos	AgendaService.marcar()	Conhecer HTTP ou JPA
Domínio (domain)	Entidades, invariantes, políticas, portas	Consulta, ConflitoHorarioPolicy, ConsultaRepository (interface)	Conhecer Spring, JPA ou HTTP
Persistência (infra)	Implementar as portas	ConsultaEntity, ConsultaMapper, ConsultaRepositoryAdapter	Conter regra de negócio
Transversal	Segurança, auditoria, erros	SecurityConfig, AuditoriaListener, @ControllerAdvice	Depender de módulos de negócio

Estrutura de pacotes (igual em todos os módulos, RNF02):

```
br.edu.ifpa.saudecerta
├── cadastro    (api, application, domain, infra)
├── agenda      (api, application, domain, infra)
├── prontuario  (api, application, domain, infra)
├── prescricao  (api, application, domain, infra)
├── relatorios  (api, application)
├── seguranca   (usuarios, perfis, auditoria)
└── shared      (config, excecoes, eventos, util)
```

Exemplo de porta definida no domínio e implementada na infra:

```
// agenda/domain
public interface ConsultaRepository {
    Consulta salvar(Consulta c);
    Optional<Consulta> buscarPorId(Long id);
            List<Consulta>   agendadasDoProfissionalNoPeriodo(Long   profissionalId,
LocalDateTime ini, LocalDateTime fim);
}
// agenda/infra: ConsultaRepositoryAdapter implements ConsultaRepository (Spring Data
JPA + mapper manual)
```

Onde cada regra é aplicada:

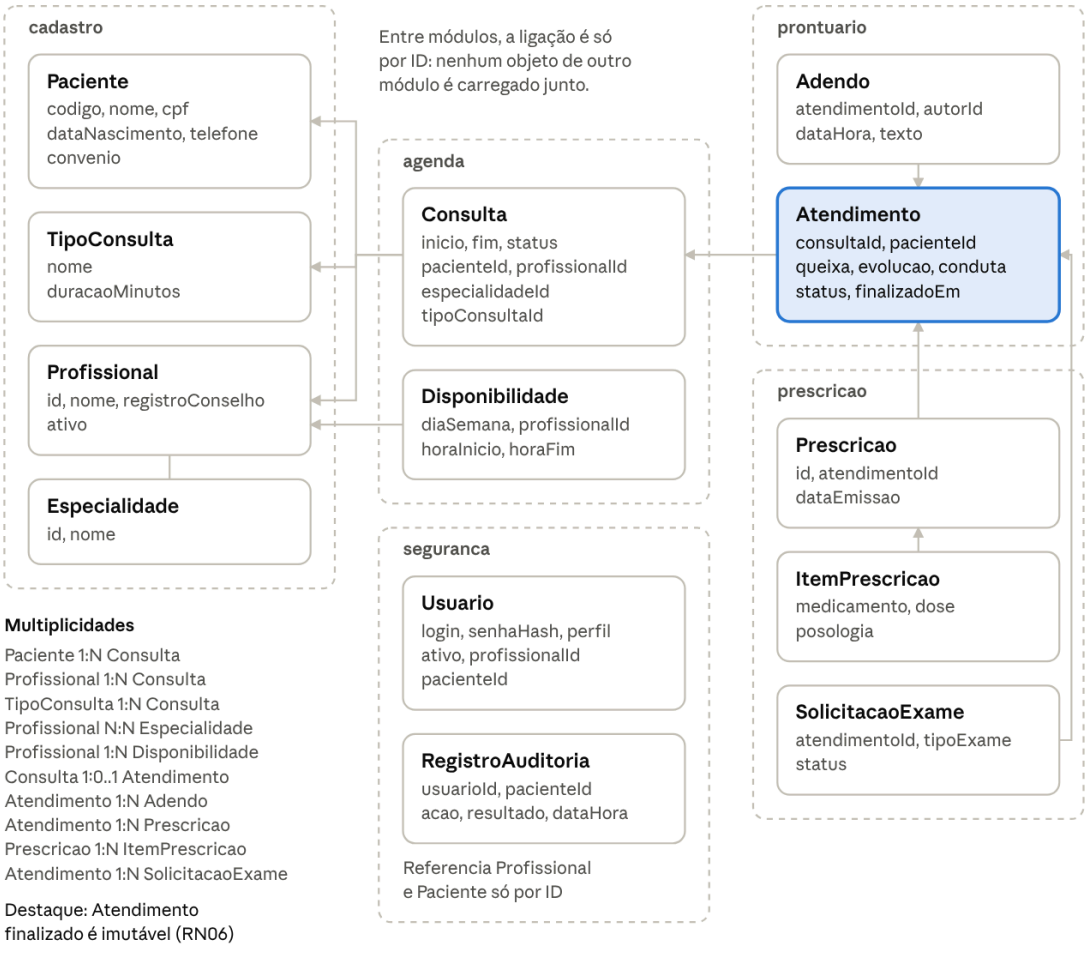
Regra	Camada e componente
RN01, RN09	Domínio (Paciente); Bean Validation na api; alerta de duplicidade no PacienteService
RN02	Domínio: ConflitoHorarioPolicy, acionada pelo AgendaService com bloqueio (ADR-004)
RN03	Domínio: Consulta calcula o fim a partir de TipoConsulta
RN04	Domínio: Consulta.podeSerAtendida(hoje)
RN05	Domínio: Consulta.remarcar() e cancelar(), com antecedência parametrizável
RN06	Domínio: Atendimento.finalizar() e adicionarAdendo(); sem setters e sem delete nos repositórios de prontuário e prescrição
RN07, RN10, RN12	Perfil na api (@PreAuthorize); vínculo e propriedade na aplicação (VinculoProfissionalPacientePolicy)
RN08	Transversal: evento e AuditoriaListener (ADR-002)
RN11	Domínio: Prescricao e SolicitacaoExame só são criadas a partir de um Atendimento em andamento

4. Modelo de domínio

Entidade	Atributos principais	Relacionamentos
Paciente	codigo (imutável), nome, cpf, dataNascimento, telefone, convenio	1 Paciente → N Consultas
Profissional	id, nome, registroConselho, ativo	N ↔ N Especialidade; 1 → N Disponibilidades
Especialidade	id, nome	N ↔ N Profissional
Disponibilidade	id, diaSemana, horaInicio, horaFim, profissionalId	N → 1 Profissional
TipoConsulta	id, nome, duracaoMinutos	1 → N Consultas
Consulta	id, dataHoraInicio, dataHoraFim, status (AGENDADA, REALIZADA, CANCELADA), pacienteId, profissionalId, especialidadeId, tipoConsultaId	1 → 0..1 Atendimento
Atendimento	id, consultaId, pacienteId, queixa, evolucao, conduta, status (EM_ANDAMENTO, FINALIZADO), finalizadoEm	1 → N Adendos, Prescrições e Solicitações de exame

Entidade	Atributos principais	Relacionamentos
Adendo	id, atendimentoId, autorId, dataHora, texto	N ⟶ 1 Atendimento
Prescricao	id, atendimentoId, dataEmissao	1 ⟶ N ItensDePrescricao
ItemPrescricao	id, medicamento, dose, posologia	N ⟶ 1 Prescricao
SolicitacaoExame	id, atendimentoId, tipoExame, status (SOLICITADO, REALIZADO)	N ⟶ 1 Atendimento
Usuario	id, login, senhaHash, perfil, ativo, profissionalId ou pacienteId (opcional)	0..1 ⟶ Profissional ou Paciente
RegistroAuditoria	id, usuarioId, pacienteId, acao (LEITURA, ALTERACAO), resultado (PERMITIDO, NEGADO), dataHora	referencia usuário e paciente por ID

Treze entidades em cinco módulos, ligadas por ID entre módulos



Multiplicidades

Paciente 1:N Consulta
Profissional 1:N Consulta
TipoConsulta 1:N Consulta
Profissional N:N Especialidade
Profissional 1:N Disponibilidade
Consulta 1:0..1 Atendimento
Atendimento 1:N Adendo
Atendimento 1:N Prescricao
Prescricao 1:N ItemPrescricao
Atendimento 1:N SolicitacaoExame

Destaque: Atendimento finalizado é imutável (RN06)

Modelo de domínio · 13 entidades em 5 módulos, ligadas por ID entre módulos

As setas indicam quem guarda a referência ao outro. O Atendimento (destacado) é o agregado central do prontuário: prescrições, exames e adendos apontam para ele.

Observações de modelagem:

- **Agregados:** Paciente; Profissional (com Disponibilidades); Consulta; Atendimento (com Adendos); Prescricao (com itens); SolicitacaoExame; Usuario; RegistroAuditoria. Entre agregados e módulos a ligação é só por ID.
- **Prontuário** é um conceito, não uma tabela: é o conjunto de Atendimentos, Adendos, Prescrições e Solicitações de exame de um paciente, lido em ordem cronológica pelo HistoricoService.
- **Especialidade na Consulta:** a consulta guarda a especialidade sob a qual foi marcada, o que permite o relatório por especialidade (RF16) dentro do módulo de agenda, sem juntar tabelas de outro módulo.
- **Convênio** é um atributo do Paciente, com 'Particular' como valor válido.
- **Invariantes:** Atendimento FINALIZADO é imutável; Adendo, Prescricao e ItemPrescricao nunca são alterados nem apagados; Consulta CANCELADA ou REALIZADA não volta a AGENDADA; o CPF é validado, mas não é chave (RN01).
- **Índices:** paciente(nome), paciente(codigo), consulta(profissionalId, dataHoraInicio), consulta(dataHoraInicio), atendimento(pacienteId, finalizadoEm), auditoria(pacienteId, dataHora).

5. Contrato da API (principais endpoints)

Método	Rota	Perfis	Requisito
POST	/api/pacientes	RECEPCAO, ADMIN	RF01
GET	/api/pacientes?nome=	RECEPCAO, ADMIN, PROFISSIONAL	RF01, RNF05
GET	/api/profissionais/{id}/agenda?data=	RECEPCAO, ADMIN, PROFISSIONAL	RF04
POST	/api/consultas	RECEPCAO, ADMIN	RF05, RF07 (409 em conflito)
PATCH	/api/consultas/{id}	RECEPCAO, ADMIN	RF06
POST	/api/consultas/{id}/atendimento	PROFISSIONAL	RF08
POST	/api/atendimentos/{id}/finalizar	PROFISSIONAL	RF08
POST	/api/atendimentos/{id}/adendos	PROFISSIONAL	RF09
POST	/api/atendimentos/{id}/prescricoes	PROFISSIONAL	RF14

Método	Rota	Perfis	Requisito
POST	/api/atendimentos/{id}/exames	PROFISSIONAL	RF15
GET	/api/pacientes/{id}/prontuario	PROFISSIONAL (vínculo), ADMIN, PACIENTE (próprio)	RF10, RF17, RF12
GET	/api/me/consultas	PACIENTE	RF17
GET	/api/relatorios/consultas?de=&ate=&agrupar=	ADMIN	RF16
GET	/api/auditoria?pacienteId=	ADMIN	RF12
POST	/api/usuarios	ADMIN	RF13

Erros padronizados pelo @ControllerAdvice: 400 (validação), 403 (sem permissão), 404 e 409 (conflito de horário).

6. Decisões arquiteturais (ADRs)

ADR-001: Arquitetura multicamadas em monólito modular

Em uma frase: um monólito dividido em camadas e módulos por domínio, em vez de microsserviços.

Status: aceita. **Drivers:** evolução e simplicidade (RNF01, RNF02).

Contexto: equipe de 3 a 4 pessoas, clínica de cerca de 20 profissionais, servidor local único. Microsserviços trariam deploy, rede e observabilidade sem benefício imediato, e o sistema precisa crescer por módulos sem reescrever o núcleo.

Decisão: três tiers (cliente, aplicação, dados); a aplicação é um monólito em quatro camadas e em módulos por domínio (cadastro, agenda, prontuário, prescrição, relatórios, segurança), com a mesma estrutura de pacotes em todos.

Alternativas:

Alternativa	Vantagem	Desvantagem
Microsserviços desde o início	Implantação e escala independentes	Complexidade operacional sem benefício nesta escala
Duas camadas (tela acessa o banco)	Muito simples	Sem controle de acesso e auditoria centralizados; fere segurança
Hexagonal completa	Isolamento máximo	Excesso de interfaces para o porte do projeto
Monólito em camadas sem módulos (pacotes por camada)	Familiar	Cada mudança se espalha por todas as camadas; extração futura difícil

Consequências: (+) entrega rápida e operação simples; (+) fronteiras prontas para extrair um serviço no Projeto Final; (−) exige disciplina para não acoplar módulos, verificada em revisão de código.

Custo/benefício: custo baixo (um deploy, um banco); o benefício principal é a modificabilidade sem o preço de serviços distribuídos.

ADR-002: Auditoria de acesso ao prontuário assíncrona, por eventos

Em uma frase: o acesso ao prontuário gera um evento, e o registro de auditoria é gravado em segundo plano.

Status: aceita. **Drivers:** RNF04, RNF05, RNF03; RF12, RN08.

Contexto: todo acesso ao prontuário deve ser registrado (RNF04), mas a abertura precisa responder em menos de 2 s (RNF05). Gravar a auditoria na mesma transação de cada leitura soma uma escrita ao caminho crítico.

Decisão: publicar o evento de domínio ProntuarioAcessado (usuário, paciente, ação LEITURA ou ALTERACAO, resultado PERMITIDO ou NEGADO, data/hora) a cada acesso, inclusive os negados, e processá-lo de forma assíncrona (@Async) no AuditoriaListener, fora do caminho crítico. Em caso de falha, o listener tenta 3 vezes e, se persistir, registra o evento em arquivo de log para reconciliação.

Alternativas:

Alternativa	Vantagem	Desvantagem
Gravar na mesma transação da leitura (síncrona)	Simples; nenhum registro perdido	Penaliza o desempenho de leitura (RNF05)
Fila externa (RabbitMQ ou Kafka)	Entrega garantida e desacoplamento	Novo componente a operar; fere a simplicidade e a stack do guia
Outbox transacional	Sem perda de evento	Mais código e tabela extra; reavaliar se o sistema crescer

Consequências: (+) preserva o desempenho de leitura; (+) desacopla a auditoria dos casos de uso; (−) risco de pequena defasagem ou, após queda do processo, de perder um registro recente; (−) exige tratar falhas do consumidor.

Custo/benefício: custo baixo (um listener e um pool de execução, sem biblioteca nova); benefício alto, pois atende RNF04 e RNF05 ao mesmo tempo.

ADR-003: Sessão com cookie e RBAC em duas etapas (perfil e vínculo)

Em uma frase: login com sessão e controle de acesso em duas etapas: primeiro o perfil, depois o vínculo com o paciente.

Status: aceita. **Drivers:** RNF03, RNF08, RN07, RN10, RN12, RF11.

Contexto: é preciso autenticar quatro perfis e garantir que o profissional só leia pacientes com quem tem vínculo e que o paciente só leia os próprios dados. Há um único servidor.

Decisão: Spring Security com login por usuário e senha (BCrypt) e sessão mantida no servidor, com cookie HttpOnly e proteção CSRF. A autorização acontece em duas etapas: (1) por perfil, com @PreAuthorize nos controllers; (2) por vínculo profissional-paciente (RN10) ou propriedade do paciente (RN12), na camada de aplicação.

Alternativas:

Alternativa	Vantagem	Desvantagem
JWT sem estado	Pronto para vários serviços	Revogação difícil; mais código e cuidado com o token
OAuth2 ou Keycloak	Robusto e padronizado	Servidor extra e curva de aprendizado; fora da stack do guia
Só perfil, sem vínculo	Mais simples	Profissional acessaria qualquer paciente, violando RN07

Consequências: (+) desativar um usuário tem efeito imediato; (+) usa só o Spring Security; (−) a sessão fica em memória e se perde ao reiniciar (o usuário entra de novo); (−) ao extrair um serviço no Projeto Final será preciso reavaliar o uso de JWT.

Custo/benefício: custo baixo; mantém a recepção ágil (RNF07) com poucos perfis e protege o dado clínico no ponto certo.

ADR-004: Conflito de horário com regra no domínio e bloqueio pessimista por profissional

Em uma frase: a regra de conflito fica no domínio, e um bloqueio por profissional evita marcação dupla simultânea.

Status: aceita. **Drivers:** RF07, RN02, RNF07.

Contexto: a checagem de sobreposição, se feita só na aplicação, falha quando duas recepcionistas marcam o mesmo horário ao mesmo tempo: ambas leem 'livre' e ambas gravam. Com 20 profissionais e várias recepcionistas isso é plausível.

Decisão: a regra (início menor que o fim existente e fim maior que o início existente, só entre consultas AGENDADAS) fica no domínio, em ConflitoHorarioPolicy. O AgendaService executa a marcação e a remarcação em uma transação que antes bloqueia a linha do profissional (@Lock PESSIMISTIC_WRITE), verifica e grava. O conflito retorna HTTP 409.

Alternativas:

Alternativa	Vantagem	Desvantagem
Só checagem na aplicação	Mais simples	Condição de corrida

Alternativa	Vantagem	Desvantagem
Constraint de exclusão no banco	Garantia no próprio banco	Recurso só do PostgreSQL; o H2 do desenvolvimento não tem, e o teste perderia a paridade
Lock otimista (versão)	Sem bloqueio	Não protege a inserção de novas linhas

Consequências: (+) garantia real contra dupla marcação no H2 e no PostgreSQL; (+) regra testável sem banco; (−) marcações do mesmo profissional são serializadas, o que não pesa nesta escala.

Custo/benefício: custo baixo (uma anotação e um teste concorrente); evita o defeito de negócio mais visível do sistema.

ADR-005: Domínio isolado da persistência, com mappers manuais

Em uma frase: o domínio é Java puro; entidades JPA e mappers manuais ficam na camada de persistência.

Status: aceita. **Drivers:** RNF01, RNF02; RN02, RN06.

Contexto: a E1 define que o domínio não conhece JPA, HTTP nem Spring Security, e o guia lista 'vazar JPA no domínio' como erro comum.

Decisão: o domínio contém classes Java puras, com comportamento (entidades ricas, sem setters públicos). Na infra existem entidades JPA próprias e mappers escritos à mão, sem biblioteca de mapeamento. Os repositórios são interfaces do domínio implementadas por adaptadores. Atendimento, Adendo, Prescrição e Solicitação de exame não têm operação de exclusão (RN06, RN11).

Alternativas:

Alternativa	Vantagem	Desvantagem
Anotações JPA direto no domínio	Menos classes	Domínio acoplado ao framework; testes exigem contexto JPA
Biblioteca de mapeamento (MapStruct)	Menos código repetitivo	Dependência fora da stack do guia
Hexagonal completa	Isolamento total	Excesso de abstração

Consequências: (+) regras RN02, RN05 e RN06 testadas em milissegundos, sem Spring nem banco; (+) fronteira limpa para extração de serviço; (−) uma classe de entidade e um mapper a mais por agregado.

Custo/benefício: custo moderado em código repetitivo, compensado pelos testes rápidos exigidos na E3 e pela facilidade de evolução.

ADR-006: Cliente web em React e banco PostgreSQL (H2 em desenvolvimento)

Em uma frase: o cliente é uma aplicação React separada da API, e o banco é PostgreSQL em produção, com H2 só no desenvolvimento.

Status: aceita. **Drivers:** RNF07, RNF02, RNF01; E1.

Contexto: a E1 define um cliente web separado, sem regra de negócio, que consome a API REST, e PostgreSQL em produção. O guia lista H2 para desenvolvimento, PostgreSQL como opcional e Postman para testar a API.

Decisão: o cliente é uma SPA em React + TypeScript (Vite), servida como arquivos estáticos, que fala só com a API. O banco é H2 em desenvolvimento e testes e PostgreSQL em produção, trocado por configuração, sem SQL específico de banco. Postman ou Insomnia continuam sendo usados para testar a API.

Alternativas:

Alternativa	Vantagem	Desvantagem
Só H2, em modo arquivo	Nada para instalar	Menos robusto com acesso concorrente; roda dentro da API, então o tier de dados não fica separado
Páginas Thymeleaf, sem SPA	Um só projeto	Mistura apresentação com o servidor e foge do cliente separado da E1
Angular no lugar de React	Estrutura mais rígida	Curva de aprendizado maior
Sem front-end, só Postman	Menos trabalho	Não permite medir a usabilidade da recepção (RNF07)

Consequências: (+) o cliente é separado e a mesma API serve interfaces futuras (mobile, portal do paciente); (+) PostgreSQL lida bem com acessos simultâneos e tem backup simples com pg_dump; (+) a usabilidade da recepção pode ser medida (RNF07); (−) exige conhecer JavaScript, TypeScript, CORS e CSRF; (−) desenvolver em H2 e rodar em PostgreSQL pode esconder diferenças de SQL, então os testes de integração finais devem rodar também contra PostgreSQL.

Custo/benefício: custo moderado (outra linguagem e um banco para instalar); o benefício é um cliente separado, como a E1 pede, e um banco adequado ao uso real da clínica.

. Matriz de rastreabilidade

Requisito	Decisão	Componente ou tática	Como verificar
RF01, RN01, RN09 (pacientes)	ADR-005	Paciente, PacienteService, Bean Validation, alerta de duplicidade	Cadastro sem campo obrigatório é recusado; CPF repetido gera alerta
RF02, RF03, RF04 (profissionais, tipos, disponibilidade)	ADR-001	Módulo cadastro; Disponibilidade;	Profissional sem especialidade é recusado;

Requisito	Decisão	Componente ou tática	Como verificar
		CalculadoraHorariosLivres	horários respeitam a duração
RF05, RF07, RN02, RN03 (marcar sem conflito)	ADR-004	ConflitoHorarioPolicy; AgendaService com lock	Teste unitário de sobreposição; teste concorrente com duas marcações
RF06, RN05 (remarcar e cancelar)	ADR-005	Consulta.remarcar() e cancelar()	Cancelar com menos de 24 h ou fora de AGENDADA é recusado
RF08, RF09, RN04, RN06 (atendimento e adendo)	ADR-005	Atendimento.finalizar(), Adendo, repositório sem delete	Editar atendimento finalizado é recusado; adendo preserva o original
RF14, RF15, RN11 (prescrição e exames)	ADR-001, ADR-005	Módulo prescricao; AtendimentoQuery	Prescrição sem atendimento em andamento é recusada
RF10, RN07, RN10, RNF03 (acesso clínico)	ADR-003	@PreAuthorize e VinculoProfissionalPacientePolicy	Acesso negado para recepção e para profissional sem vínculo
RF17, RN12 (paciente)	ADR-003	Perfil PACIENTE e regra de propriedade	Paciente lê o próprio prontuário e recebe 403 no de outro
RF11, RF13 (perfis e usuários)	ADR-003	Spring Security, UsuarioService	Testes por perfil em cada endpoint
RF12, RN08, RNF04 (auditoria)	ADR-002	ProntuarioAcessado, AuditoriaListener	Registro existe após abrir prontuário; negações também são registradas
RF16 (relatórios)	ADR-001	Módulo relatorios; ConsultasAgregadasQuery	Totais conferem com dados de teste; só ADMIN acessa
RNF01 (modificabilidade)	ADR-001, ADR-005	Módulos por domínio, portas, eventos	Revisão de código: sem ciclos nem acesso ao repositório de outro módulo
RNF02 (manutenibilidade)	ADR-001, ADR-006	Estrutura de pacotes única, H2, README	Colega novo sobe o sistema em até 30 min

Requisito	Decisão	Componente ou tática	Como verificar
RNF05 (desempenho)	ADR-002	Índices, paginação, auditoria assíncrona	Medir o p95 com a carga de teste (20 profissionais, 10 mil pacientes, 50 mil consultas)
RNF06 (disponibilidade)	ADR-001	Reinício automático, Spring Actuator, transações curtas	Matar o processo e medir o retorno; consultar /actuator/health
RNF07 (usabilidade)	ADR-006, ADR-004	SPA com fluxo UC01 curto; erro 409 claro	Cronometrar e contar cliques no fluxo de marcação
RNF08 (credenciais)	ADR-003	BCrypt no UsuarioService	Revisão de código; checar a coluna senhaHash
RNF09 (recuperabilidade)	ADR-006	Script pg_dump agendado, com cópia	Restaurar o backup em outro ambiente uma vez por mês